

SOL-Lite — Operational Build v0.1

The existing prototype already gives us the important seed: workspace management, architecture-drift checking, file operations, terminal execution, code search, human approval gating, fault logging, tool definitions, and the Ollama loop.

We are **not** going to keep growing that single script.

The target is a proper application where every major subsystem can be tested independently.

1. Complete SOL-Lite directory structure

This is the target structure I want us to build toward.

```
SOL-Lite/
├── README.md
├── LICENSE
├── CHANGELOG.md
├── pyproject.toml
├── requirements.txt
├── .gitignore
├── .env.example
├── Makefile
├── config/
│   ├── sol-lite.yaml
│   ├── permissions.yaml
│   ├── agents.yaml
│   ├── models.yaml
│   ├── schedules.yaml
│   └── logging.yaml
├── scripts/
│   ├── install.sh
│   ├── uninstall.sh
│   ├── start.sh
│   ├── stop.sh
│   ├── doctor.sh
│   ├── test.sh
│   └── reset_dev.sh
├── src/
│   ├── sol_lite/
│   │   ├── __init__.py
│   │   ├── __main__.py
│   │   ├── app.py
│   │   ├── bootstrap.py
│   │   └── version.py
│   ├── core/
│   │   ├── __init__.py
│   │   ├── runtime.py
│   │   └── event_bus.py
```

- lifecycle.py
- health.py
- exceptions.py

- agents/

- __init__.py
 - base.py
 - registry.py
 - manager.py
 - context.py

- pa/
 - __init__.py
 - agent.py
 - prompt.md
 - tools.yaml

- business/
 - __init__.py
 - agent.py
 - prompt.md
 - tools.yaml

- docs/
 - __init__.py
 - agent.py
 - prompt.md
 - tools.yaml

- engineer/
 - __init__.py
 - agent.py
 - prompt.md
 - tools.yaml

- models/

- __init__.py
 - base.py
 - ollama.py
 - manager.py
 - profiles.py
 - health.py

- tools/

- __init__.py
 - base.py
 - registry.py
 - executor.py
 - filesystem.py
 - terminal.py
 - search.py
 - processes.py
 - git.py
 - applications.py

- permissions/

- `__init__.py`
- `engine.py`
- `policy.py`
- `scopes.py`
- `approvals.py`
- `dangerous.py`

- `memory/`
 - `__init__.py`
 - `manager.py`
 - `global_memory.py`
 - `project_memory.py`
 - `agent_memory.py`
 - `lessons.py`
 - `embeddings.py`
 - `rag.py`

- `workspace/`
 - `__init__.py`
 - `manager.py`
 - `projects.py`
 - `files.py`
 - `metadata.py`
 - `indexing.py`

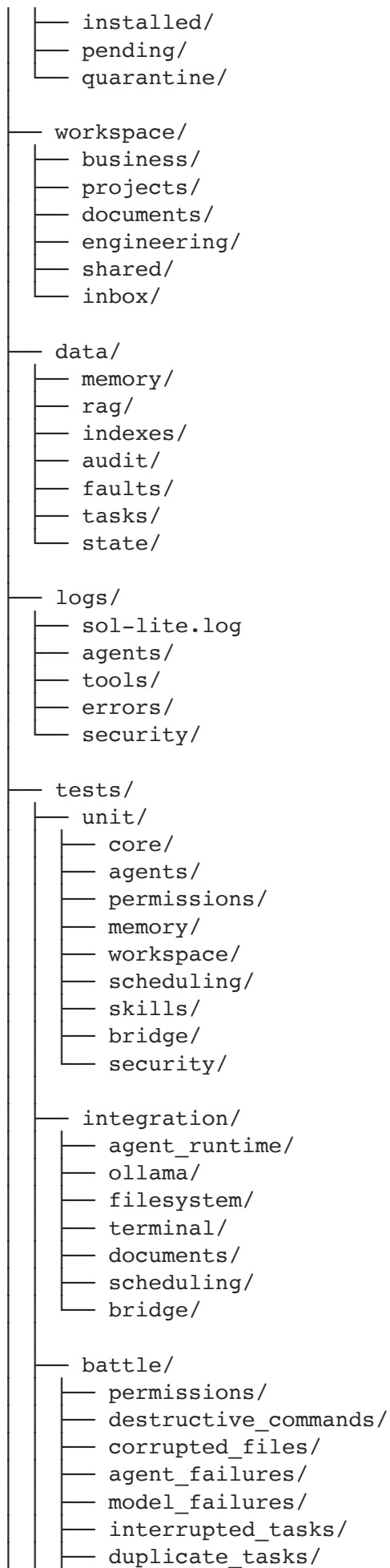
- `documents/`
 - `__init__.py`
 - `classifier.py`
 - `metadata.py`
 - `renamer.py`
 - `organiser.py`
 - `indexer.py`
 - `pipeline.py`

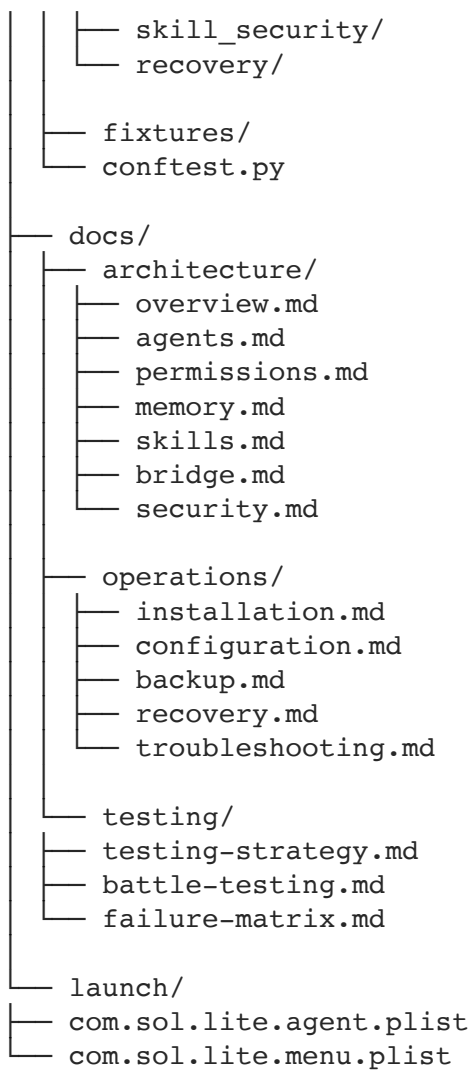
- `scheduling/`
 - `__init__.py`
 - `manager.py`
 - `tasks.py`
 - `recurrence.py`
 - `priorities.py`
 - `time_tracking.py`
 - `briefings.py`

- `skills/`
 - `__init__.py`
 - `manifest.py`
 - `inspector.py`
 - `installer.py`
 - `registry.py`
 - `permissions.py`
 - `sandbox.py`

- `integrations/`
 - `__init__.py`
 - `macos/`
 - `calendar.py`
 - `reminders.py`

- mail.py
- contacts.py
- notes.py
- notifications.py
- finder.py
- browsers/
 - safari.py
 - chrome.py
- developer/
 - vscode.py
 - git.py
 - github.py
- nas/
 - discovery.py
 - storage.py
 - infrastructure.py
- bridge/
 - __init__.py
 - protocol.py
 - client.py
 - server.py
 - discovery.py
 - schemas.py
- audit/
 - __init__.py
 - logger.py
 - events.py
 - activity.py
 - audit_store.py
- security/
 - __init__.py
 - secrets.py
 - path_guard.py
 - command_guard.py
 - integrity.py
- ui/
 - __init__.py
 - cli.py
 - menu_bar.py
 - desktop.py
 - voice.py
- agents/
 - sol-pa/
 - sol-business/
 - sol-docs/
 - sol-engineer/
- skills/
 - builtin/





2. What we're building first

We are **not** going to write 150 files blindly.
The build order will be:

```
FOUNDATION
↓
CORE RUNTIME
↓
PERMISSIONS + AUDIT
↓
TOOLS
↓
AGENT RUNTIME
↓
SOL PA
↓
SOL BUSINESS
↓
SOL DOCS
↓
SOL ENGINEER
```

↓
MEMORY / RAG
↓
SCHEDULING
↓
DOCUMENT PIPELINE
↓
SKILLS
↓
MACOS INTEGRATIONS
↓
SOL BRIDGE
↓
UI
↓
BATTLE TESTING

And importantly:

Every layer gets tests before we build the next layer.

That is how we avoid ending up with another 400-line script that technically works but becomes difficult to trust.

3. Section 1 — Foundation

Create:

```
SOL-Lite/  
├── pyproject.toml  
├── requirements.txt  
├── .gitignore  
├── .env.example  
├── Makefile  
  
├── config/  
│   ├── sol-lite.yaml  
│   ├── permissions.yaml  
│   ├── agents.yaml  
│   └── models.yaml  
  
├── src/  
├── sol_lite/  
├── __init__.py  
├── __main__.py  
├── version.py  
├── app.py  
└── bootstrap.py
```

FILE 1 —

pyproject.toml

```

[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "sol-lite"
version = "0.1.0"
description = "SOL-Lite – local-first multi-agent assistant for macOS"
readme = "README.md"
requires-python = ">=3.11"
license = { text = "Proprietary" }

dependencies = [
    "PyYAML>=6.0",
    "pydantic>=2.0",
    "httpx>=0.27",
    "rich>=13.0",
    "python-dotenv>=1.0",
]

[project.optional-dependencies]
test = [
    "pytest>=8.0",
    "pytest-asyncio>=0.23",
    "pytest-cov>=5.0",
]

dev = [
    "ruff>=0.6",
    "mypy>=1.10",
    "pytest>=8.0",
    "pytest-asyncio>=0.23",
    "pytest-cov>=5.0",
]

[project.scripts]
sol-lite = "sol_lite.app:main"

[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["src"]
asyncio_mode = "auto"

[tool.ruff]
line-length = 100
target-version = "py311"

[tool.mypy]
python_version = "3.11"
warn_return_any = true
warn_unused_configs = true
disallow_untyped_defs = true

```

FILE 2 —

requirements.txt

```
PyYAML>=6.0
pydantic>=2.0
httpx>=0.27
rich>=13.0
python-dotenv>=1.0

pytest>=8.0
pytest-asyncio>=0.23
pytest-cov>=5.0
```

FILE 3 —

.gitignore

```
# Python
__pycache__/
*.py[co]
*.so
*.egg-info/
.pytest_cache/
.mypy_cache/
.ruff_cache/
.coverage
htmlcov/

# Virtual environments
.venv/
venv/
env/

# Environment
.env

# macOS
.DS_Store

# IDE
.idea/
.vscode/
```

```
# SOL-Lite runtime data
data/
logs/
workspace/
skills/installed/
skills/pending/
skills/quarantine/

# Local model/runtime state
*.sqlite
*.sqlite3
*.db

# Secrets
*.key
*.pem
*.p12
*.pfx
secrets/
```

FILE 4 —

.env.example

```
# SOL-Lite environment configuration

SOL_LITE_ENV=development

# Ollama
SOL_OLLAMA_URL=http://127.0.0.1:11434
SOL_DEFAULT_MODEL=qwen2.5-coder:7b

# Runtime
SOL_LOG_LEVEL=INFO
SOL_WORKING_HOURS_START=08:00
SOL_WORKING_HOURS_END=18:00

# After-hours operation
SOL_AFTER_HOURS_ENABLED=false

# Security
SOL_REQUIRE_APPROVAL_FOR_WRITES=true
SOL_REQUIRE_APPROVAL_FOR_SHELL=true

# Workspace
SOL_WORKSPACE_ROOT=./workspace

# Data
SOL_DATA_ROOT=./data

# Optional NAS configuration
```

```
# SOL_NAS_ROOT=/Volumes/YOUR_NAS/SOL
```

FILE 5 —

Makefile

```
.PHONY: install test lint format typecheck check run doctor clean

install:
python3 -m venv .venv
. .venv/bin/activate && pip install --upgrade pip
. .venv/bin/activate && pip install -r requirements.txt

test:
. .venv/bin/activate && pytest -v

test-cov:
. .venv/bin/activate && pytest -v --cov=sol_lite --cov-report=term-missing

lint:
. .venv/bin/activate && ruff check src tests

format:
. .venv/bin/activate && ruff format src tests

typecheck:
. .venv/bin/activate && mypy src

check:
$(MAKE) lint
$(MAKE) typecheck
$(MAKE) test

run:
. .venv/bin/activate && python -m sol_lite

doctor:
. .venv/bin/activate && python -m sol_lite doctor

clean:
find . -type d -name "__pycache__" -prune -exec rm -rf {} \;
find . -type d -name ".pytest_cache" -prune -exec rm -rf {} \;
find . -type d -name ".mypy_cache" -prune -exec rm -rf {} \;
```

FILE 6 —

config/sol-lite.yaml

```
application:
name: SOL-Lite
version: "0.1.0"
environment: development

runtime:
mode: assisted

# Assisted:
# Agent proposes actions and asks for approval where required.
#
# Autonomous:
# Agent may execute actions permitted by policy.
#
# Scheduled:
# Background/after-hours execution is permitted.

max_concurrent_agents: 4
max_tool_calls_per_task: 50
task_timeout_seconds: 900

workspace:
root: "./workspace"

directories:
business: "./workspace/business"
projects: "./workspace/projects"
documents: "./workspace/documents"
engineering: "./workspace/engineering"
shared: "./workspace/shared"
inbox: "./workspace/inbox"

data:
root: "./data"

directories:
memory: "./data/memory"
rag: "./data/rag"
indexes: "./data/indexes"
audit: "./data/audit"
faults: "./data/faults"
tasks: "./data/tasks"
state: "./data/state"

logging:
level: INFO
console: true
file: "./logs/sol-lite.log"

background:
enabled: false
allow_after_hours: false
```

```
macos:
integrations_enabled: false

nas:
enabled: false
path: null

bridge:
enabled: false
listen_host: "127.0.0.1"
listen_port: 8787
```

FILE 7 —

config/permissions.yaml

This is deliberately conservative.
The original prototype has a single HUMAN_APPROVED gate for writes.
SQL-Lite replaces that with **operation-specific permissions**.

```
permissions:

filesystem:
read:
enabled: true
approval_required: false

write:
enabled: true
approval_required: true

move:
enabled: true
approval_required: true

delete:
enabled: false
approval_required: true

terminal:
enabled: true
approval_required: true

allowed_commands:
- python3
- python
- git
- ls
- pwd
- find
- grep
```

- rg
- cat
- head
- tail
- mkdir
- cp
- mv
- touch
- which
- whoami

blocked_commands:

- rm
- rmdir
- sudo
- shutdown
- reboot
- diskutil
- mkfs
- chmod
- chown

network:

enabled: false

approval_required: true

applications:

enabled: false

approval_required: true

macos:

enabled: false

approval_required: true

skills:

inspect:

enabled: true

approval_required: false

install:

enabled: true

approval_required: true

execute:

enabled: false

approval_required: true

bridge:

enabled: false

approval_required: true

FILE 8 —

config/agents.yaml

agents:

sol_pa:

name: SOL PA

enabled: true

role: primary

description: >

Primary user-facing assistant responsible for coordination, planning, scheduling, communication, research, and delegation.

capabilities:

- planning
- scheduling
- prioritisation
- research
- delegation
- web_development
- frontend_development
- backend_development

can_delegate_to:

- sol_business
- sol_docs
- sol_engineer

sol_business:

name: SOL Business

enabled: true

role: specialist

description: >

Business operations, projects, reporting, planning, administrative workflows, and operational analysis.

capabilities:

- business_operations
- project_management
- reporting
- analysis
- administration

can_delegate_to:

- sol_docs
- sol_engineer

sol_docs:

name: SOL Docs

enabled: true

role: specialist

description: >

Document ingestion, classification, metadata, organisation, indexing, knowledge extraction, and document workflows.

```

capabilities:
- document_classification
- metadata
- organisation
- indexing
- extraction
- rag

can_delegate_to: []

sol_engineer:
name: SOL Engineer
enabled: true
role: specialist
description: >
Full-stack software engineering, automation, debugging,
architecture, testing, development, and technical operations.

capabilities:
- software_engineering
- frontend
- backend
- full_stack
- automation
- debugging
- testing
- architecture
- devops
- git

can_delegate_to:
- sol_docs

```

FILE 9 —

config/models.yaml

The model is intentionally configurable.
We do **not** want SOL-Lite permanently welded to the current prototype's:

```
TARGET_MODEL = 'qwen2.5-coder:7b'
```

That line exists in the prototype today.
Instead:

```

provider:
type: ollama

endpoint: "http://127.0.0.1:11434"

profiles:

```

```
default:
model: "qwen2.5-coder:7b"
temperature: 0.2
timeout_seconds: 300
```

```
pa:
model: "qwen2.5-coder:7b"
temperature: 0.3
timeout_seconds: 300
```

```
business:
model: "qwen2.5-coder:7b"
temperature: 0.3
timeout_seconds: 300
```

```
docs:
model: "qwen2.5-coder:7b"
temperature: 0.1
timeout_seconds: 300
```

```
engineer:
model: "qwen2.5-coder:7b"
temperature: 0.15
timeout_seconds: 600
```

```
fallbacks:
enabled: true
models: []
```

Later we can add:

```
fast model
reasoning model
coding model
document model
embedding model
```

without redesigning the agent architecture.

FILE 10 —

src/sol_lite/__init__.py

```
"""
SOL-Lite

A local-first, multi-agent assistant for macOS.
"""

from .version import __version__

__all__ = ["__version__"]
```

FILE 11 —

src/sol_lite/version.py

```
"""SOL-Lite version information."""  
  
__version__ = "0.1.0"
```

FILE 12 —

src/sol_lite/__main__.py

```
"""Allow SOL-Lite to be launched with:  
  
python -m sol_lite  
"""  
  
from .app import main  
  
if __name__ == "__main__":  
    main()
```

FILE 13 —

src/sol_lite/app.py

This is deliberately tiny.
The application entry point should **not** contain the agent logic.

```
"""SOL-Lite application entry point."""  
  
from __future__ import annotations  
  
import argparse  
import sys  
  
from .bootstrap import bootstrap
```

```

def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        prog="sol-lite",
        description="SOL-Lite local-first multi-agent assistant",
    )

    subparsers = parser.add_subparsers(dest="command")

    subparsers.add_parser(
        "start",
        help="Start the SOL-Lite runtime.",
    )

    subparsers.add_parser(
        "doctor",
        help="Run local health and configuration checks.",
    )

    subparsers.add_parser(
        "version",
        help="Display SOL-Lite version.",
    )

    return parser


def main() -> None:
    parser = build_parser()
    args = parser.parse_args()

    if args.command == "version":
        from .version import __version__

        print(f"SOL-Lite {__version__}")
        return

    if args.command == "doctor":
        runtime = bootstrap()
        result = runtime.health_check()

        if result.ok:
            print("SOL-Lite doctor: OK")
            sys.exit(0)

        print("SOL-Lite doctor: FAILED")

        for error in result.errors:
            print(f" ERROR: {error}")

        sys.exit(1)

    if args.command == "start":
        runtime = bootstrap()
        runtime.start()
        return

```

```
parser.print_help()
```

```
if __name__ == "__main__":  
    main()
```

FILE 14 —

src/sol_lite/bootstrap.py

This is our first important architectural boundary.

```
"""SOL-Lite runtime bootstrap."""
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass  
from pathlib import Path
```

```
import yaml
```

```
from .core.health import HealthResult  
from .core.runtime import Runtime
```

```
@dataclass  
class RuntimeConfig:  
    application: dict  
    runtime: dict  
    workspace: dict  
    data: dict  
    logging: dict  
    background: dict  
    macos: dict  
    nas: dict  
    bridge: dict
```

```
def load_yaml(path: Path) -> dict:  
    if not path.exists():  
        raise FileNotFoundError(f"Configuration file not found: {path}")
```

```
    with path.open("r", encoding="utf-8") as handle:  
        data = yaml.safe_load(handle)
```

```
    if not isinstance(data, dict):  
        raise ValueError(f"Configuration must contain a YAML object: {path}")
```

```
    return data
```

```

def load_config() -> RuntimeConfig:
    root = Path.cwd()
    config_dir = root / "config"

    application = load_yaml(config_dir / "sol-lite.yaml")

    return RuntimeConfig(
        application=application.get("application", {}),
        runtime=application.get("runtime", {}),
        workspace=application.get("workspace", {}),
        data=application.get("data", {}),
        logging=application.get("logging", {}),
        background=application.get("background", {}),
        macos=application.get("macos", {}),
        nas=application.get("nas", {}),
        bridge=application.get("bridge", {}),
    )

def bootstrap() -> Runtime:
    config = load_config()

    runtime = Runtime(config=config)

    return runtime

```

4. Section 2 — Core runtime

Next we create:

```

src/sol_lite/core/
├── __init__.py
├── runtime.py
├── event_bus.py
├── lifecycle.py
├── health.py
└── exceptions.py

```

This is where SOL-Lite stops being "a script that talks to Ollama" and becomes an actual runtime.

FILE 15 —

src/sol_lite/core/__init__.py

```

"""SOL-Lite core runtime."""

from .health import HealthResult
from .runtime import Runtime

```

```
__all__ = [  
    "HealthResult",  
    "Runtime",  
]
```

FILE 16 —

src/sol_lite/core/exceptions.py

```
"""SOL-Lite exception hierarchy."""  
  
class SOLLiteError(Exception):  
    """Base exception for SOL-Lite."""  
  
class ConfigurationError(SOLLiteError):  
    """Raised when configuration is invalid."""  
  
class PermissionDenied(SOLLiteError):  
    """Raised when an operation is not permitted."""  
  
class ApprovalRequired(SOLLiteError):  
    """Raised when an operation requires human approval."""  
  
class ToolExecutionError(SOLLiteError):  
    """Raised when a tool fails during execution."""  
  
class AgentError(SOLLiteError):  
    """Raised when an agent fails."""  
  
class ModelError(SOLLiteError):  
    """Raised when the model provider fails."""  
  
class TaskError(SOLLiteError):  
    """Raised when a task cannot complete."""  
  
class SecurityError(SOLLiteError):  
    """Raised when a security boundary is violated."""
```

FILE 17 —

src/sol_lite/core/health.py

```
"""Runtime health reporting."""

from __future__ import annotations

from dataclasses import dataclass, field

@dataclass
class HealthResult:
    ok: bool
    errors: list[str] = field(default_factory=list)
    warnings: list[str] = field(default_factory=list)

    @classmethod
    def success(cls) -> "HealthResult":
        return cls(ok=True)

    @classmethod
    def failure(
        cls,
        errors: list[str],
        warnings: list[str] | None = None,
    ) -> "HealthResult":
        return cls(
            ok=False,
            errors=errors,
            warnings=warnings or [],
        )
```

FILE 18 —

src/sol_lite/core/event_bus.py

Every important action will eventually emit an event.

This is what will power:

- activity screen
- audit trail
- debugging
- agent-to-agent visibility
- background operation
- future UI
- SOL Command bridge
- SOL Outpost bridge

```
"""Internal SOL-Lite event bus."""
```

```

from __future__ import annotations

from collections import defaultdict
from collections.abc import Callable
from dataclasses import dataclass, field
from datetime import datetime, timezone
from typing import Any
from uuid import uuid4


@dataclass
class Event:
    type: str
    source: str
    data: dict[str, Any] = field(default_factory=dict)

    event_id: str = field(default_factory=lambda: str(uuid4()))
    timestamp: datetime = field(
        default_factory=lambda: datetime.now(timezone.utc)
    )


EventHandler = Callable[[Event], None]


class EventBus:
    def __init__(self) -> None:
        self._handlers: dict[str, list[EventHandler]] = defaultdict(list)

    def subscribe(
        self,
        event_type: str,
        handler: EventHandler,
    ) -> None:
        self._handlers[event_type].append(handler)

    def publish(self, event: Event) -> None:
        handlers = self._handlers.get(event.type, [])

        for handler in handlers:
            handler(event)

    def clear(self) -> None:
        self._handlers.clear()

```

FILE 19 —

src/sol_lite/core/lifecycle.py

```

"""SOL-Lite runtime lifecycle state."""

```

```

from __future__ import annotations

from enum import Enum


class RuntimeState(str, Enum):
    CREATED = "created"
    STARTING = "starting"
    RUNNING = "running"
    STOPPING = "stopping"
    STOPPED = "stopped"
    FAILED = "failed"

```

FILE 20 —

src/sol_lite/core/runtime.py

```

"""SOL-Lite central runtime."""

from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path

from .event_bus import Event, EventBus
from .health import HealthResult
from .lifecycle import RuntimeState


@dataclass
class Runtime:
    config: object

    def __post_init__(self) -> None:
        self.state = RuntimeState.CREATED
        self.events = EventBus()

    @property
    def application_name(self) -> str:
        application = getattr(self.config, "application", {})
        return application.get("name", "SOL-Lite")

    @property
    def version(self) -> str:
        application = getattr(self.config, "application", {})
        return application.get("version", "unknown")

    def start(self) -> None:
        if self.state == RuntimeState.RUNNING:
            return

```

```

self.state = RuntimeState.STARTING

self.events.publish(
Event(
type="runtime.starting",
source="core.runtime",
data={
"application": self.application_name,
"version": self.version,
},
)
)

self._ensure_directories()

self.state = RuntimeState.RUNNING

self.events.publish(
Event(
type="runtime.started",
source="core.runtime",
data={
"application": self.application_name,
"version": self.version,
},
)
)

print(
f"{self.application_name} {self.version} "
f"runtime started."
)

def stop(self) -> None:
if self.state != RuntimeState.RUNNING:
return

self.state = RuntimeState.STOPPING

self.events.publish(
Event(
type="runtime.stopping",
source="core.runtime",
)
)

self.state = RuntimeState.STOPPED

self.events.publish(
Event(
type="runtime.stopped",
source="core.runtime",
)
)

def health_check(self) -> HealthResult:

```

```

errors: list[str] = []
warnings: list[str] = []

if not getattr(self.config, "application", None):
errors.append("Application configuration is missing.")

workspace = getattr(self.config, "workspace", {})
workspace_root = workspace.get("root")

if not workspace_root:
errors.append("Workspace root is not configured.")
else:
path = Path(workspace_root)

if not path.exists():
warnings.append(
f"Workspace root does not exist yet: {path}"
)

if errors:
return HealthResult.failure(
errors=errors,
warnings=warnings,
)

return HealthResult(
ok=True,
warnings=warnings,
)

def _ensure_directories(self) -> None:
workspace = getattr(self.config, "workspace", {})
data = getattr(self.config, "data", {})

self._create_configured_directories(
workspace.get("directories", {})
)

self._create_configured_directories(
data.get("directories", {})
)

@staticmethod
def _create_configured_directories(
directories: dict,
) -> None:
for directory in directories.values():
Path(directory).mkdir(
parents=True,
exist_ok=True,
)

```

SOL-Lite — Operational Build v0.1

The existing prototype already gives us the important seed: workspace management, architecture-drift checking, file operations, terminal execution, code search, human approval gating, fault logging, tool definitions, and the Ollama loop.

We are **not** going to keep growing that single script.

The target is a proper application where every major subsystem can be tested independently.

1. Complete SOL-Lite directory structure

This is the target structure I want us to build toward.

```
SOL-Lite/
├── README.md
├── LICENSE
├── CHANGELOG.md
├── pyproject.toml
├── requirements.txt
├── .gitignore
├── .env.example
├── Makefile
├── config/
│   ├── sol-lite.yaml
│   ├── permissions.yaml
│   ├── agents.yaml
│   ├── models.yaml
│   ├── schedules.yaml
│   └── logging.yaml
├── scripts/
│   ├── install.sh
│   ├── uninstall.sh
│   ├── start.sh
│   ├── stop.sh
│   ├── doctor.sh
│   ├── test.sh
│   └── reset_dev.sh
├── src/
│   ├── sol_lite/
│   │   ├── __init__.py
│   │   ├── __main__.py
│   │   ├── app.py
│   │   ├── bootstrap.py
│   │   └── version.py
│   └── core/
│       ├── __init__.py
│       ├── runtime.py
│       ├── event_bus.py
│       ├── lifecycle.py
│       ├── health.py
│       └── exceptions.py
```

```
— agents/
  — __init__.py
  — base.py
  — registry.py
  — manager.py
  — context.py

  — pa/
    — __init__.py
    — agent.py
    — prompt.md
    — tools.yaml

  — business/
    — __init__.py
    — agent.py
    — prompt.md
    — tools.yaml

  — docs/
    — __init__.py
    — agent.py
    — prompt.md
    — tools.yaml

  — engineer/
    — __init__.py
    — agent.py
    — prompt.md
    — tools.yaml

— models/
  — __init__.py
  — base.py
  — ollama.py
  — manager.py
  — profiles.py
  — health.py

— tools/
  — __init__.py
  — base.py
  — registry.py
  — executor.py

  — filesystem.py
  — terminal.py
  — search.py
  — processes.py
  — git.py
  — applications.py

— permissions/
  — __init__.py
  — engine.py
  — policy.py
```

- scopes.py
- approvals.py
- dangerous.py

- memory/
 - __init__.py
 - manager.py
 - global_memory.py
 - project_memory.py
 - agent_memory.py
 - lessons.py
 - embeddings.py
 - rag.py

- workspace/
 - __init__.py
 - manager.py
 - projects.py
 - files.py
 - metadata.py
 - indexing.py

- documents/
 - __init__.py
 - classifier.py
 - metadata.py
 - renamer.py
 - organiser.py
 - indexer.py
 - pipeline.py

- scheduling/
 - __init__.py
 - manager.py
 - tasks.py
 - recurrence.py
 - priorities.py
 - time_tracking.py
 - briefings.py

- skills/
 - __init__.py
 - manifest.py
 - inspector.py
 - installer.py
 - registry.py
 - permissions.py
 - sandbox.py

- integrations/
 - __init__.py
 - macos/
 - calendar.py
 - reminders.py
 - mail.py
 - contacts.py
 - notes.py

- └─ notifications.py
- └─ finder.py

- └─ browsers/
 - └─ safari.py
 - └─ chrome.py

- └─ developer/
 - └─ vscode.py
 - └─ git.py
 - └─ github.py

- └─ nas/
 - └─ discovery.py
 - └─ storage.py
 - └─ infrastructure.py

- └─ bridge/
 - └─ __init__.py
 - └─ protocol.py
 - └─ client.py
 - └─ server.py
 - └─ discovery.py
 - └─ schemas.py

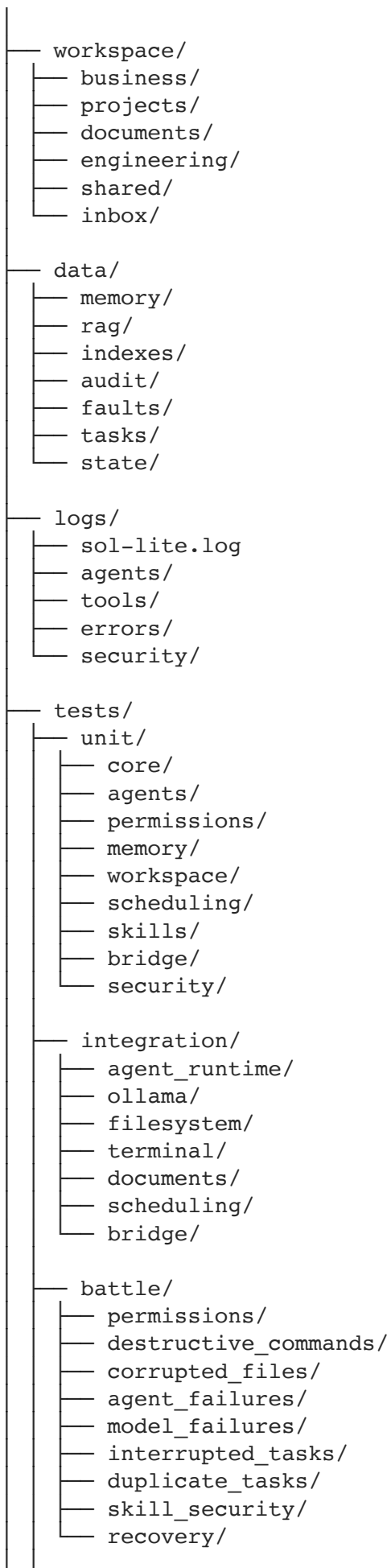
- └─ audit/
 - └─ __init__.py
 - └─ logger.py
 - └─ events.py
 - └─ activity.py
 - └─ audit_store.py

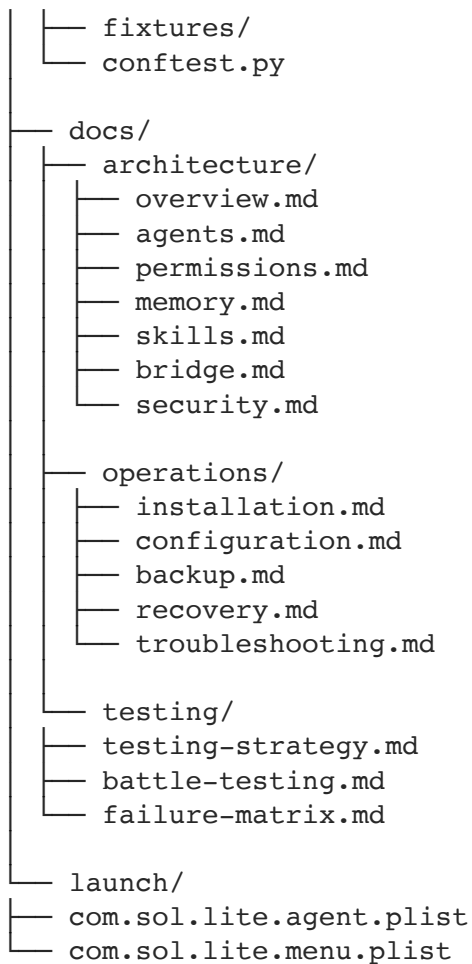
- └─ security/
 - └─ __init__.py
 - └─ secrets.py
 - └─ path_guard.py
 - └─ command_guard.py
 - └─ integrity.py

- └─ ui/
 - └─ __init__.py
 - └─ cli.py
 - └─ menu_bar.py
 - └─ desktop.py
 - └─ voice.py

- └─ agents/
 - └─ sol-pa/
 - └─ sol-business/
 - └─ sol-docs/
 - └─ sol-engineer/

- └─ skills/
 - └─ builtin/
 - └─ installed/
 - └─ pending/
 - └─ quarantine/





2. What we're building first

We are **not** going to write 150 files blindly.
The build order will be:

```
FOUNDATION
↓
CORE RUNTIME
↓
PERMISSIONS + AUDIT
↓
TOOLS
↓
AGENT RUNTIME
↓
SOL PA
↓
SOL BUSINESS
↓
SOL DOCS
↓
SOL ENGINEER
↓
MEMORY / RAG
↓
```

SCHEDULING
↓
DOCUMENT PIPELINE
↓
SKILLS
↓
MACOS INTEGRATIONS
↓
SOL BRIDGE
↓
UI
↓
BATTLE TESTING

And importantly:

Every layer gets tests before we build the next layer.

That is how we avoid ending up with another 400-line script that technically works but becomes difficult to trust.

3. Section 1 — Foundation

Create:

```
SOL-Lite/  
├── pyproject.toml  
├── requirements.txt  
├── .gitignore  
├── .env.example  
├── Makefile  
  
├── config/  
│   ├── sol-lite.yaml  
│   ├── permissions.yaml  
│   ├── agents.yaml  
│   └── models.yaml  
  
├── src/  
├── sol_lite/  
├── __init__.py  
├── __main__.py  
├── version.py  
├── app.py  
└── bootstrap.py
```

FILE 1 —

pyproject.toml

[build-system]

```

requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "sol-lite"
version = "0.1.0"
description = "SOL-Lite – local-first multi-agent assistant for macOS"
readme = "README.md"
requires-python = ">=3.11"
license = { text = "Proprietary" }

dependencies = [
    "PyYAML>=6.0",
    "pydantic>=2.0",
    "httpx>=0.27",
    "rich>=13.0",
    "python-dotenv>=1.0",
]

[project.optional-dependencies]
test = [
    "pytest>=8.0",
    "pytest-asyncio>=0.23",
    "pytest-cov>=5.0",
]

dev = [
    "ruff>=0.6",
    "mypy>=1.10",
    "pytest>=8.0",
    "pytest-asyncio>=0.23",
    "pytest-cov>=5.0",
]

[project.scripts]
sol-lite = "sol_lite.app:main"

[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["src"]
asyncio_mode = "auto"

[tool.ruff]
line-length = 100
target-version = "py311"

[tool.mypy]
python_version = "3.11"
warn_return_any = true
warn_unused_configs = true
disallow_untyped_defs = true

```

FILE 2 —

requirements.txt

```
PyYAML>=6.0
pydantic>=2.0
httpx>=0.27
rich>=13.0
python-dotenv>=1.0

pytest>=8.0
pytest-asyncio>=0.23
pytest-cov>=5.0
```

FILE 3 —

.gitignore

```
# Python
__pycache__/
*.py[cod]
*.so
*.egg-info/
.pytest_cache/
.mypy_cache/
.ruff_cache/
.coverage
htmlcov/

# Virtual environments
.venv/
venv/
env/

# Environment
.env

# macOS
.DS_Store

# IDE
.idea/
.vscode/

# SOL-Lite runtime data
data/
```

```
logs/
workspace/
skills/installed/
skills/pending/
skills/quarantine/

# Local model/runtime state
*.sqlite
*.sqlite3
*.db

# Secrets
*.key
*.pem
*.p12
*.pfx
secrets/
```

FILE 4 —

.env.example

```
# SOL-Lite environment configuration

SOL_LITE_ENV=development

# Ollama
SOL_OLLAMA_URL=http://127.0.0.1:11434
SOL_DEFAULT_MODEL=qwen2.5-coder:7b

# Runtime
SOL_LOG_LEVEL=INFO
SOL_WORKING_HOURS_START=08:00
SOL_WORKING_HOURS_END=18:00

# After-hours operation
SOL_AFTER_HOURS_ENABLED=false

# Security
SOL_REQUIRE_APPROVAL_FOR_WRITES=true
SOL_REQUIRE_APPROVAL_FOR_SHELL=true

# Workspace
SOL_WORKSPACE_ROOT=./workspace

# Data
SOL_DATA_ROOT=./data

# Optional NAS configuration
# SOL_NAS_ROOT=/Volumes/YOUR_NAS/SOL
```

FILE 5 —

Makefile

```
.PHONY: install test lint format typecheck check run doctor clean

install:
python3 -m venv .venv
. .venv/bin/activate && pip install --upgrade pip
. .venv/bin/activate && pip install -r requirements.txt

test:
. .venv/bin/activate && pytest -v

test-cov:
. .venv/bin/activate && pytest -v --cov=sol_lite --cov-report=term-missing

lint:
. .venv/bin/activate && ruff check src tests

format:
. .venv/bin/activate && ruff format src tests

typecheck:
. .venv/bin/activate && mypy src

check:
$(MAKE) lint
$(MAKE) typecheck
$(MAKE) test

run:
. .venv/bin/activate && python -m sol_lite

doctor:
. .venv/bin/activate && python -m sol_lite doctor

clean:
find . -type d -name "__pycache__" -prune -exec rm -rf {} \;
find . -type d -name ".pytest_cache" -prune -exec rm -rf {} \;
find . -type d -name ".mypy_cache" -prune -exec rm -rf {} \;
```

FILE 6 —

config/sol-lite.yaml

```
application:
name: SOL-Lite
version: "0.1.0"
environment: development

runtime:
mode: assisted

# Assisted:
# Agent proposes actions and asks for approval where required.
#
# Autonomous:
# Agent may execute actions permitted by policy.
#
# Scheduled:
# Background/after-hours execution is permitted.

max_concurrent_agents: 4
max_tool_calls_per_task: 50
task_timeout_seconds: 900

workspace:
root: "./workspace"

directories:
business: "./workspace/business"
projects: "./workspace/projects"
documents: "./workspace/documents"
engineering: "./workspace/engineering"
shared: "./workspace/shared"
inbox: "./workspace/inbox"

data:
root: "./data"

directories:
memory: "./data/memory"
rag: "./data/rag"
indexes: "./data/indexes"
audit: "./data/audit"
faults: "./data/faults"
tasks: "./data/tasks"
state: "./data/state"

logging:
level: INFO
console: true
file: "./logs/sol-lite.log"

background:
enabled: false
allow_after_hours: false

macos:
integrations_enabled: false
```

```
nas:
enabled: false
path: null

bridge:
enabled: false
listen_host: "127.0.0.1"
listen_port: 8787
```

FILE 7 —

config/permissions.yaml

This is deliberately conservative.
The original prototype has a single HUMAN_APPROVED gate for writes.
SQL-Lite replaces that with **operation-specific permissions**.

```
permissions:

filesystem:
read:
enabled: true
approval_required: false

write:
enabled: true
approval_required: true

move:
enabled: true
approval_required: true

delete:
enabled: false
approval_required: true

terminal:
enabled: true
approval_required: true

allowed_commands:
- python3
- python
- git
- ls
- pwd
- find
- grep
- rg
- cat
- head
- tail
```

- mkdir
- cp
- mv
- touch
- which
- whoami

blocked_commands:

- rm
- rmdir
- sudo
- shutdown
- reboot
- diskutil
- mkfs
- chmod
- chown

network:

enabled: false
approval_required: true

applications:

enabled: false
approval_required: true

macos:

enabled: false
approval_required: true

skills:

inspect:
enabled: true
approval_required: false

install:

enabled: true
approval_required: true

execute:

enabled: false
approval_required: true

bridge:

enabled: false
approval_required: true

FILE 8 —

config/agents.yaml

agents:

sol_pa:

name: SOL PA

enabled: true

role: primary

description: >

Primary user-facing assistant responsible for coordination, planning, scheduling, communication, research, and delegation.

capabilities:

- planning
- scheduling
- prioritisation
- research
- delegation
- web_development
- frontend_development
- backend_development

can_delegate_to:

- sol_business
- sol_docs
- sol_engineer

sol_business:

name: SOL Business

enabled: true

role: specialist

description: >

Business operations, projects, reporting, planning, administrative workflows, and operational analysis.

capabilities:

- business_operations
- project_management
- reporting
- analysis
- administration

can_delegate_to:

- sol_docs
- sol_engineer

sol_docs:

name: SOL Docs

enabled: true

role: specialist

description: >

Document ingestion, classification, metadata, organisation, indexing, knowledge extraction, and document workflows.

capabilities:

- document_classification
- metadata
- organisation
- indexing

```

- extraction
- rag

can_delegate_to: []

sol_engineer:
name: SOL Engineer
enabled: true
role: specialist
description: >
Full-stack software engineering, automation, debugging,
architecture, testing, development, and technical operations.

capabilities:
- software_engineering
- frontend
- backend
- full_stack
- automation
- debugging
- testing
- architecture
- devops
- git

can_delegate_to:
- sol_docs

```

FILE 9 —

config/models.yaml

The model is intentionally configurable.
We do **not** want SOL-Lite permanently welded to the current prototype's:

```
TARGET_MODEL = 'qwen2.5-coder:7b'
```

That line exists in the prototype today.
Instead:

```

provider:
type: ollama

endpoint: "http://127.0.0.1:11434"

profiles:

default:
model: "qwen2.5-coder:7b"
temperature: 0.2
timeout_seconds: 300

```

```
pa:
model: "qwen2.5-coder:7b"
temperature: 0.3
timeout_seconds: 300
```

```
business:
model: "qwen2.5-coder:7b"
temperature: 0.3
timeout_seconds: 300
```

```
docs:
model: "qwen2.5-coder:7b"
temperature: 0.1
timeout_seconds: 300
```

```
engineer:
model: "qwen2.5-coder:7b"
temperature: 0.15
timeout_seconds: 600
```

```
fallbacks:
enabled: true
models: []
```

Later we can add:

```
fast model
reasoning model
coding model
document model
embedding model
```

without redesigning the agent architecture.

FILE 10 —

src/sol_lite/__init__.py

```
"""
SOL-Lite

A local-first, multi-agent assistant for macOS.
"""

from .version import __version__

__all__ = ["__version__"]
```

FILE 11 —

src/sol_lite/version.py

```
"""SOL-Lite version information."""

__version__ = "0.1.0"
```

FILE 12 —

src/sol_lite/__main__.py

```
"""Allow SOL-Lite to be launched with:

python -m sol_lite
"""

from .app import main

if __name__ == "__main__":
    main()
```

FILE 13 —

src/sol_lite/app.py

This is deliberately tiny.
The application entry point should **not** contain the agent logic.

```
"""SOL-Lite application entry point."""

from __future__ import annotations

import argparse
import sys

from .bootstrap import bootstrap

def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        prog="sol-lite",
```

```

description="SQL-Lite local-first multi-agent assistant",
)

subparsers = parser.add_subparsers(dest="command")

subparsers.add_parser(
    "start",
    help="Start the SQL-Lite runtime.",
)

subparsers.add_parser(
    "doctor",
    help="Run local health and configuration checks.",
)

subparsers.add_parser(
    "version",
    help="Display SQL-Lite version.",
)

return parser

def main() -> None:
    parser = build_parser()
    args = parser.parse_args()

    if args.command == "version":
        from .version import __version__

        print(f"SQL-Lite {__version__}")
        return

    if args.command == "doctor":
        runtime = bootstrap()
        result = runtime.health_check()

        if result.ok:
            print("SQL-Lite doctor: OK")
            sys.exit(0)

        print("SQL-Lite doctor: FAILED")

    for error in result.errors:
        print(f" ERROR: {error}")

    sys.exit(1)

    if args.command == "start":
        runtime = bootstrap()
        runtime.start()
        return

    parser.print_help()

if __name__ == "__main__":

```

```
main()
```

FILE 14 —

src/sol_lite/bootstrap.py

This is our first important architectural boundary.

```
"""SOL-Lite runtime bootstrap."""

from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path

import yaml

from .core.health import HealthResult
from .core.runtime import Runtime


@dataclass
class RuntimeConfig:
    application: dict
    runtime: dict
    workspace: dict
    data: dict
    logging: dict
    background: dict
    macos: dict
    nas: dict
    bridge: dict


def load_yaml(path: Path) -> dict:
    if not path.exists():
        raise FileNotFoundError(f"Configuration file not found: {path}")

    with path.open("r", encoding="utf-8") as handle:
        data = yaml.safe_load(handle)

    if not isinstance(data, dict):
        raise ValueError(f"Configuration must contain a YAML object: {path}")

    return data


def load_config() -> RuntimeConfig:
    root = Path.cwd()
    config_dir = root / "config"

    application = load_yaml(config_dir / "sol-lite.yaml")
```

```

return RuntimeConfig(
    application=application.get("application", {}),
    runtime=application.get("runtime", {}),
    workspace=application.get("workspace", {}),
    data=application.get("data", {}),
    logging=application.get("logging", {}),
    background=application.get("background", {}),
    macos=application.get("macos", {}),
    nas=application.get("nas", {}),
    bridge=application.get("bridge", {}),
)

```

```

def bootstrap() -> Runtime:
    config = load_config()

    runtime = Runtime(config=config)

    return runtime

```

4. Section 2 — Core runtime

Next we create:

```

src/sol_lite/core/
├── __init__.py
├── runtime.py
├── event_bus.py
├── lifecycle.py
├── health.py
└── exceptions.py

```

This is where SOL-Lite stops being "a script that talks to Ollama" and becomes an actual runtime.

FILE 15 —

src/sol_lite/core/__init__.py

```

"""SOL-Lite core runtime."""

from .health import HealthResult
from .runtime import Runtime

__all__ = [
    "HealthResult",
    "Runtime",
]

```

FILE 16 —

src/sol_lite/core/exceptions.py

```
"""SOL-Lite exception hierarchy."""

class SOLLiteError(Exception):
    """Base exception for SOL-Lite."""

class ConfigurationError(SOLLiteError):
    """Raised when configuration is invalid."""

class PermissionDenied(SOLLiteError):
    """Raised when an operation is not permitted."""

class ApprovalRequired(SOLLiteError):
    """Raised when an operation requires human approval."""

class ToolExecutionError(SOLLiteError):
    """Raised when a tool fails during execution."""

class AgentError(SOLLiteError):
    """Raised when an agent fails."""

class ModelError(SOLLiteError):
    """Raised when the model provider fails."""

class TaskError(SOLLiteError):
    """Raised when a task cannot complete."""

class SecurityError(SOLLiteError):
    """Raised when a security boundary is violated."""
```

FILE 17 —

src/sol_lite/core/health.py

```

"""Runtime health reporting."""

from __future__ import annotations

from dataclasses import dataclass, field

@dataclass
class HealthResult:
    ok: bool
    errors: list[str] = field(default_factory=list)
    warnings: list[str] = field(default_factory=list)

    @classmethod
    def success(cls) -> "HealthResult":
        return cls(ok=True)

    @classmethod
    def failure(
        cls,
        errors: list[str],
        warnings: list[str] | None = None,
    ) -> "HealthResult":
        return cls(
            ok=False,
            errors=errors,
            warnings=warnings or [],
        )

```

FILE 18 —

src/sol_lite/core/event_bus.py

Every important action will eventually emit an event.
This is what will power:

- activity screen
- audit trail
- debugging
- agent-to-agent visibility
- background operation
- future UI
- SOL Command bridge
- SOL Outpost bridge

```

"""Internal SOL-Lite event bus."""

from __future__ import annotations

from collections import defaultdict
from collections.abc import Callable
from dataclasses import dataclass, field

```

```

from datetime import datetime, timezone
from typing import Any
from uuid import uuid4

@dataclass
class Event:
    type: str
    source: str
    data: dict[str, Any] = field(default_factory=dict)

    event_id: str = field(default_factory=lambda: str(uuid4()))
    timestamp: datetime = field(
        default_factory=lambda: datetime.now(timezone.utc)
    )

EventHandler = Callable[[Event], None]

class EventBus:
    def __init__(self) -> None:
        self._handlers: dict[str, list[EventHandler]] = defaultdict(list)

    def subscribe(
        self,
        event_type: str,
        handler: EventHandler,
    ) -> None:
        self._handlers[event_type].append(handler)

    def publish(self, event: Event) -> None:
        handlers = self._handlers.get(event.type, [])

        for handler in handlers:
            handler(event)

    def clear(self) -> None:
        self._handlers.clear()

```

FILE 19 —

src/sol_lite/core/lifecycle.py

```

"""SOL-Lite runtime lifecycle state."""

from __future__ import annotations

from enum import Enum

```

```

class RuntimeState(str, Enum):
    CREATED = "created"
    STARTING = "starting"
    RUNNING = "running"
    STOPPING = "stopping"
    STOPPED = "stopped"
    FAILED = "failed"

```

FILE 20 —

src/sol_lite/core/runtime.py

```

"""SOL-Lite central runtime."""

from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path

from .event_bus import Event, EventBus
from .health import HealthResult
from .lifecycle import RuntimeState

@dataclass
class Runtime:
    config: object

    def __post_init__(self) -> None:
        self.state = RuntimeState.CREATED
        self.events = EventBus()

    @property
    def application_name(self) -> str:
        application = getattr(self.config, "application", {})
        return application.get("name", "SOL-Lite")

    @property
    def version(self) -> str:
        application = getattr(self.config, "application", {})
        return application.get("version", "unknown")

    def start(self) -> None:
        if self.state == RuntimeState.RUNNING:
            return

        self.state = RuntimeState.STARTING

        self.events.publish(
            Event(
                type="runtime.starting",

```

```

source="core.runtime",
data={
  "application": self.application_name,
  "version": self.version,
},
)
)

self._ensure_directories()

self.state = RuntimeState.RUNNING

self.events.publish(
  Event(
    type="runtime.started",
    source="core.runtime",
    data={
      "application": self.application_name,
      "version": self.version,
    },
  ),
)

print(
  f"{self.application_name} {self.version} "
  f"runtime started."
)

def stop(self) -> None:
  if self.state != RuntimeState.RUNNING:
    return

  self.state = RuntimeState.STOPPING

  self.events.publish(
    Event(
      type="runtime.stopping",
      source="core.runtime",
    ),
  )

  self.state = RuntimeState.STOPPED

  self.events.publish(
    Event(
      type="runtime.stopped",
      source="core.runtime",
    ),
  )

def health_check(self) -> HealthResult:
  errors: list[str] = []
  warnings: list[str] = []

  if not getattr(self.config, "application", None):
    errors.append("Application configuration is missing.")

```

```

workspace = getattr(self.config, "workspace", {})
workspace_root = workspace.get("root")

if not workspace_root:
    errors.append("Workspace root is not configured.")
else:
    path = Path(workspace_root)

    if not path.exists():
        warnings.append(
            f"Workspace root does not exist yet: {path}"
        )

    if errors:
        return HealthResult.failure(
            errors=errors,
            warnings=warnings,
        )

    return HealthResult(
        ok=True,
        warnings=warnings,
    )

def _ensure_directories(self) -> None:
    workspace = getattr(self.config, "workspace", {})
    data = getattr(self.config, "data", {})

    self._create_configured_directories(
        workspace.get("directories", {})
    )

    self._create_configured_directories(
        data.get("directories", {})
    )

    @staticmethod
    def _create_configured_directories(
        directories: dict,
    ) -> None:
        for directory in directories.values():
            Path(directory).mkdir(
                parents=True,
                exist_ok=True,
            )

```